LINGLONG SG98

New Motor Backend Implementation Template

A copy-and-fill playbook for adding a new RS-485 motor family by replacing only the motor-specific commands.

Keep RT shape. Replace only the motor-specific blocks.

Primary source: Documentation/New-Motor-Backend-Implementation-Template.md

Copy RT And Keep The Shape

The fastest path is to preserve the existing loop structure and only swap the drive-specific register logic.

Keep

Disconnect handling, comm health checks, state updates, send and receive flow, and the split between connect, run, stop, and polling.

Replace

Probe register, setup writes, forward and reverse commands, telemetry decode, stop path, sleep path, and optional reset path.

Use JMC only when needed

Switch to the JMC backend as your template only when you need multi-baud probing or a more stateful enable sequence.

Template starting point: src/RT/motorcontrol_rt.cpp

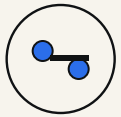
Collect These Manual Facts Before Coding

The backend goes quickly once the safe registers and command model are known.



Bus

Baud, parity, stop bits, slave ID rules, and whether the drive boots in a predictable profile.



Control

Enable, disable, run, stop, direction, speed, torque, ramp, save, sleep, and reset commands.



Telemetry

Status, actual RPM, actual torque or load, and a fault register that can be trusted during bench testing.

If the worksheet is incomplete, stop and get the missing manual data first

Know Which Comm Helper To Use

The transport layer is already done. The main choice is whether the command is blocking bring-up work or async runtime traffic.

Blocking helpers

Use writeConfirm, writeMultipleConfirm, and readReturn for probe, one-time setup, off, sleep, and reset.

Async sends

Use writeSingleRegister, writeMultipleRegisters, and readRegister inside tick for the live comm loop.

Receive path

Call receive in tick, then decode lastRequestType and lastRead so the backend updates cached mirrors and live telemetry.

Transport truth: src/comm.cpp

Top Of File: Registers, Mirrors, And Probe

Start the new backend by declaring readable constants and local register mirrors instead of editing the RT numbers in place.

Constants

Add FOO_REG_STATUS, FOO_REG_RPM, FOO_REG_TORQUE, enable, direction, ramp, fault, and any split forward or reverse speed registers.

Mirrors

Keep confirmed reg_* mirrors and pending_reg_* values for async multi-register writes so tick knows what still needs to be sent.

Probe helper

Build a tiny probe around motor_probe_set_profile plus motor_probe_read_register, and make it specific enough that it cannot lie.

Use readable names tied to motor function, not recycled RT register numbers

Fill Probe, Init, And Connect-Time Setup

init should only clear local state. The actual drive setup belongs in the first connected branch inside tick.

probe

Return true only when a distinctive status block really answers. For multi-baud families, follow the JMC try-and-restore pattern.

init

Reset motor_setRPM, live readback, commStatus, currentStatus, and every cached reg_* mirror. Keep init lightweight.

First connection

Inside the commStatus == COMM_DISCONNECTED branch, send one-time mode, source, ramp, or save writes that must happen once per connection.

Heavy setup goes in the connect-time branch, not in init()

The Tick Function Has Four Replaceable Blocks

The send and receive loop already has a good shape. Most ports only need to replace the motor-specific logic inside each block.

Receive decode

Decode write acknowledgements, read responses, and optional multi-write responses. Update reg_* mirrors, currentStatus, motor_currentRPM, and motor_currentTorque.

Stop branch

When effectiveSetRPM is zero, write the drive to a safe stopped state in the exact order the manual expects, then switch into polling.

Run branches

Mirror the same pattern for forward and reverse: set enable, torque, speed, and direction or jog mode, then poll once the drive is settled.

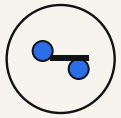
Choose The Right Command Pattern

Most new motors fit one of three models: split forward and reverse speed registers, absolute speed plus direction, or absolute speed plus signed torque.



RT style

Forward speed and reverse speed are separate registers. Forward writes speed_fwd, reverse writes speed_rev, and jog mode selects direction.



Direction register style

The drive wants one absolute speed register and a separate direction value. Forward and reverse both set the same speed register.



Signed torque style

The drive uses one absolute speed register but torque or command polarity determines the real direction, similar to the JMC path.

Pick the model that matches the manual, then keep forward and reverse logic symmetric

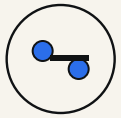
Wire The Backend Into Motor HAL

Once the new file compiles, the rest is table work: DriveType, probe report fields, command descriptors, and the MotorDescriptor entry.



DriveType

Add DRIVE_TYPE_FOO in src/motor_hal.h and only the MotorProbeReport fields needed to explain what the probe matched.



Command table

Define FOO_COMMANDS with menuNum, itemIdx, valueScale, supported, and help text. Mark unsupported controls honestly.



MotorDescriptor

Register key, label, commProfile, maxRpm, torquePercentNumerator, probePriority, commands, and MOTOR_BACKEND_FOO.

Shared registration lives in src/motor_hal.h and src/motor_hal.cpp

Finish With Scaling And Bring-Up Order

Most bugs after the first spin are not architecture bugs. They are usually command scaling, torque decode, or test-order mistakes.

Scaling

Use valueScale to turn UI values into drive units. Start by copying the closest existing backend, then bench-check displayed torque percent.

Comm profile

Add a new CommProfile only if baud, parity, stop bits, or timing truly differ. Otherwise reuse the closest existing profile.

Bench order

Compile, probe, off, status, RPM readback, forward run, reverse run, torque, then optional ramp, save, sleep, and reset.

Lowest-risk rule: make off() work before you chase full-speed run control